

LAPORAN PRAKTEK KECERDASAN BUATAN

Pertemuan Ke-5 : Program Algoritma Pencarian



Nama : Aura Sasi Kirana Dharma Acintya

NRP : 3123500048

Dosen Pengajar : Nur Rosyid Muftadai S.Kom., M.T

**PROGRAM STUDI D3 TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)**

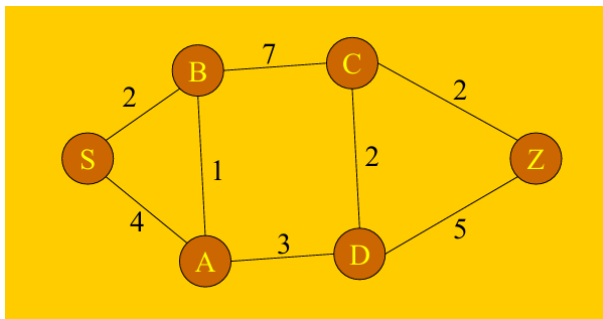
TAHUN 2025

A. Tugas

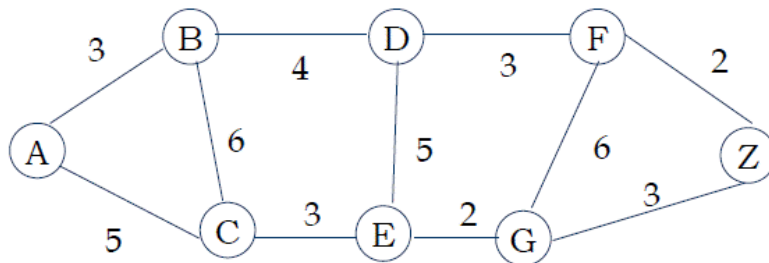
Buatlah kode program untuk algoritma pencarian, menggunakan metode:

- Breadth First Search
- Depth First Search
- Best First Search
- Hill Climbing
- Branch and Bound
- Dynamic Programming

a. Tugas di PPT



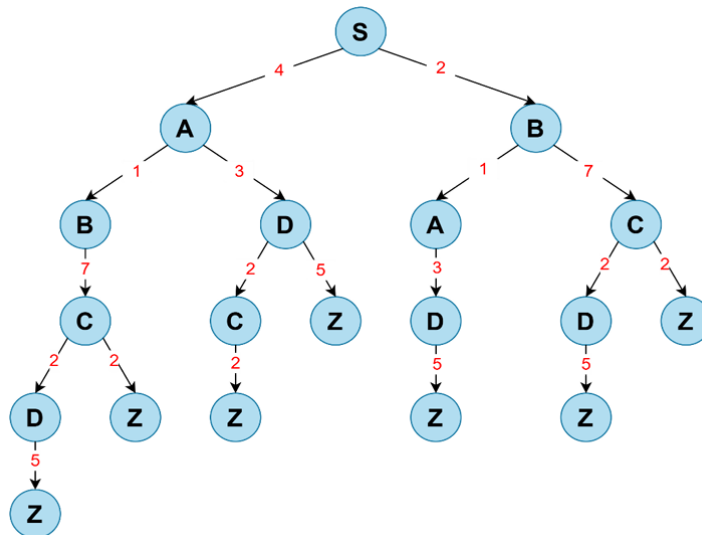
b. Tugas di Lampiran



B. Jawaban dan Solusi

a. Tugas di PPT

Gambar Diagram Tree



- Breadth First Search

```
# **1. Breadth First Search (BFS)**
def bfs(graph, start, goal):
    queue = [[start]]

    while queue:
        path = queue.pop(0)
        node = path[-1]

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in path:
                new_path = path + [neighbor]
                queue.append(new_path)

print("BFS Path", bfs(graph, "S", "Z"))
```

Analisa:

- **Graph** dipresentasikan sebagai direktori, setiap kuncinya adalah node, dan nilai berupa daftar node yang terhubung sebagai tetangga (neighbor).
- **`Def bsf(graph, start, goal)`** merupakan fungsi untuk menerapkan algoritma Breath First Search untuk menemukan jalur dari start (S) ke goal (Z), yaitu mencari jalur terpendek dalam graf tanpa bobot.
- **Queue** (antrian) digunakan untuk menyimpan jalur yang sedang di cari

dalam BFS.

- **Path** adalah daftar node yang membentuk jalur dari start (S) hingga node yang diperiksa. Jika `path[-1]` adalah goal (Z), maka jalur akan dikembalikan sebagai hasil
- ``node = path[-1]`` artinya mengambil simpul terakhir dalam jalur yang sedang diproses. Jika ``node == goal``, maka jalur ditemukan dan dikembalikan.
- **Neighbor** adalah node yang terhubung dengan node dalam graph untuk mengeksplorasi jalur baru.
- **BFS** (Breadth First Search) efektif untuk mencari jalur terpendek dalam graf tanpa bobot menggunakan queue untuk eksplorasi level demi level.

- Depth First Search

```
# **2. Depth First Search (DFS)**
def dfs(graph, start, goal, path=[]):
    path.append(start)

    if start == goal:
        return path

    for neighbor in graph.get(start, []):
        if neighbor not in path:
            new_path = dfs(graph, neighbor, goal, path.copy())
            if new_path:
                return new_path

print("DFS Path:", dfs(graph, "S", "Z"))
```

Analisa:

- ``def dfs(graph, start, goal, path=[])`` menerapkan algoritma Depth-First Search untuk mencari jalur dari start (S) ke goal (Z), yaitu mencari jalur secara mendalam (Deep) sebelum berpindah ke jalur lain.
- **DFS** tidak menggunakan queue, tetapi menggunakan rekursi untuk mencari jalur.
- **Path** digunakan untuk memastikan node tidak dikunjungi dua kali dan untuk melacak jalur.
- **Node** adalah simpul yang sedang diperiksa, jika ``start == goal``, jalur akan dikembalikan sebagai hasil pencarian.
- **DFS** (Depth First Search) adalah cara untuk mencari jalur menggunakan

rekursi, yaitu menelusuri jalur secara mendalam terlebih dahulu sebelum kembali mencoba jalur lain (jika perlu).

- Best First Search

```
# **3. Best First Search (Tanpa Library Sorting)**
def best_first_search(graph, start, goal, heuristic):
    queue = [(heuristic[start], [start])]

    while queue:
        # **Sorting manual tanpa fungsi sort()**
        for i in range(len(queue) - 1):
            for j in range(i + 1, len(queue)):
                if queue[i][0] > queue[j][0]: # Tukar jika lebih besar
                    queue[i], queue[j] = queue[j], queue[i]

        _, path = queue.pop(0)
        node = path[-1]

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in path:
                new_path = path + [neighbor]
                queue.append((heuristic[neighbor], new_path))

    heuristic = {"S": 6, "A": 5, "B": 4, "C": 2, "D": 3, "Z": 0}
    print("Best First Search Path:", best_first_search(graph, "S", "Z", heuristic))
```

Analisa:

- `def best_first_search(graph, start, goal, heuristic)` untuk mencari jalur dari start ke goal untuk memilih jalur yang dianggap paling menjanjikan terlebih dahulu.
- Queue menyimpan pasangan (heuristic, path), agar daftar antrian diurutkan berdasarkan nilai heuristic terkecil sehingga proses sorting dilakukan manual tanpa fungsi `sort()`.
- **Heuristic** (Estimasi Jarak ke goal) adalah nilai perkiraan yang menunjukkan seberapa dekat suatu node ke goal. **Node** dengan nilai heuristic lebih kecil akan diprioritaskan.
- **Path** menyimpan daftar node untuk memastikan node tidak dikunjungi 2 kali.
- **Node** adalah node terakhir dalam path. Jika `node == goal` maka jalur akan dikembalikan sebagai hasil pencarian.
- **Best First Search** adalah cara mencari jalur optimal berdasarkan heuristic terkecil terlebih dahulu, tetapi tidak menjamin jalur yang ditemukan adalah jalur terpendek.

- Hill Climbing

```
# **4. Hill Climbing (Tanpa Library Sorting)**
def hill_climbing(graph, start, goal, heuristic):
    path = [start]
    current = start

    while current != goal:
        neighbors = graph.get(current, [])
        if not neighbors:
            return None

        # **Mencari minimum secara manual tanpa sort()**
        next_node = neighbors[0]
        for node in neighbors:
            if heuristic[node] < heuristic[next_node]:
                next_node = node

        if heuristic[next_node] >= heuristic[current]: # Tidak ada perbaikan
            return None

        path.append(next_node)
        current = next_node

    return path

print("Hill Climbing Path:", hill_climbing(graph, "S", "Z", heuristic))
```

Analisa:

- `def hill_climbing(graph, start, goal, heuristic)` adalah algoritma pencarian yang selalu memilih neighbor terbaik berdasarkan heuristic.
- **Current** digunakan untuk menyimpan node yang sedang dievaluasi. Berubah setiap iterasi ke neighbor terbaik berdasarkan nilai heuristic.
- **Neighbor** berisi daftar node yang terhubung langsung dengan current. Dicari node dengan heuristic terendah untuk dijadikan langkah berikutnya.
- **Heuristic** digunakan untuk memilih neighbor terbaik dengan nilai heuristic lebih kecil dari current.
- **Path** menyimpan urutan node dari start hingga solusi atau jalur berhenti. Jika mencapai goal (Z), path dikembalikan sebagai hasil pencarian
- **Node** dalam perulangan adalah setiap neighbor dari current. Algoritma membandingkan nilai heuristic untuk menentukan langkah terbaik.
- **Hill Climbing** adalah cara mencari jalur ke goal (Z) dengan memilih neighbor terbaik di setiap langkah. Namun, metode ini bisa terjebak di lokal optimum jika tidak ada neighbor yang lebih baik dari current.

- Branch and Bound

```
# **5. Branch and Bound (Tanpa Sorting Bawaan)**
def branch_and_bound(graph, start, goal):
    queue = [(0, [start])]

    while queue:
        # **Sorting manual tanpa fungsi sort()**
        for i in range(len(queue) - 1):
            for j in range(i + 1, len(queue)):
                if queue[i][0] > queue[j][0]: # Tukar jika lebih besar
                    queue[i], queue[j] = queue[j], queue[i]

        cost, path = queue.pop(0)
        node = path[-1]

        if node == goal:
            return path, cost

        for neighbor, weight in graph.get(node, {}).items():
            if neighbor not in path:
                queue.append((cost + weight, path + [neighbor]))

weighted_graph = {
    "S": {"A": 4, "B": 2},
    "A": {"B": 1, "D": 3},
    "B": {"C": 7},
    "C": {"D": 2, "Z": 2},
    "D": {"Z": 5}
}

print("Branch and Bound Path:", branch_and_bound(weighted_graph, "S", "Z"))
```

Analisa:

- **Weighted_graph** adalah graph yang memiliki bobot sebagai direktori nested, dimana setiap node memiliki neighbor dan bobot (cost) masing-masing.
- ``def branch_and_bound(graph, start, goal)`` adalah pencarian jalur terpendek berdasarkan biaya kumulatif, menggunakan antrian prioritas manual dengan sorting dari jalur dengan bobot terkecil terlebih dahulu.
- **Queue** digunakan untuk menyimpan (cost, path), dimana cost adalah total bobot jalur, dan path adalah jalur yang diikuti. Jalur dengan cost terkecil akan dilewati terlebih dahulu.
- **Range** digunakan untuk mengurutkan jalur berdasarkan cost secara manual tanpa ``sort()``.
- **Cost** adalah jumlah bobot jalur yang telah diambil. Saat menambahkan neighbor baru, cost diperbarui dengan cost baru + weight edge.
- **Weight** adalah bobot antar node terbaru dengan neighbornya dalam graf berbobot.
- **Neighbor** adalah key dari direktori yang menyimpan node yang bisa

dikunjungi dari node terbaru.

- **Path** menyimpan urutan node dari start hingga result. ``node = path[-1]`` mengambil node terakhir dalam jalur saat ini yang sedang diperiksa.
 - **Branch and Bound** adalah algoritma pencarian dengan antrian prioritas menggunakan cost terendah. Sorting dilakukan secara manual untuk memastikan jalur dengan cost terkecil akan di lalui terlebih dahulu.
- Dynamic Programming

```
def dijkstra(graph, start, goal):
    unvisited = {node: float("inf") for node in graph}
    unvisited[start] = 0
    visited = {}
    path = {}

    while unvisited:
        current_node = min(unvisited, key=unvisited.get)

        print(f"Memproses node: {current_node}, dengan biaya: {unvisited[current_node]}")

        if current_node == goal:
            break

        for neighbor, cost in graph[current_node]:
            if neighbor in visited:
                continue

            new_cost = unvisited[current_node] + cost
            if new_cost < unvisited.get(neighbor, float("inf")):
                unvisited[neighbor] = new_cost
                path[neighbor] = current_node

        visited[current_node] = unvisited[current_node]
        del unvisited[current_node]

    print("Path tracking:", path)

    # Rekonstruksi jalur terpendek
    final_path = []
    node = goal
    while node in path:
        final_path.insert(0, node)
        node = path[node]

    if node == start:
        final_path.insert(0, start)
        return final_path, visited[goal] if goal in visited else "Jalur tidak ditemukan"
    else:
        return "Jalur tidak ditemukan"

# Graph berbobot yang diperbaiki
weighted_graph = {
    "S": [("A", 4), ("B", 2)],
    "A": [("B", 1), ("D", 3)],
    "B": [("C", 7)],
    "C": [("D", 2), ("Z", 2)],
    "D": [("Z", 5)],
    "Z": [] # Tambahkan agar dikenali
}

print("Dijkstra Path:", dijkstra(weighted_graph, "S", "Z"))
```

Analisa:

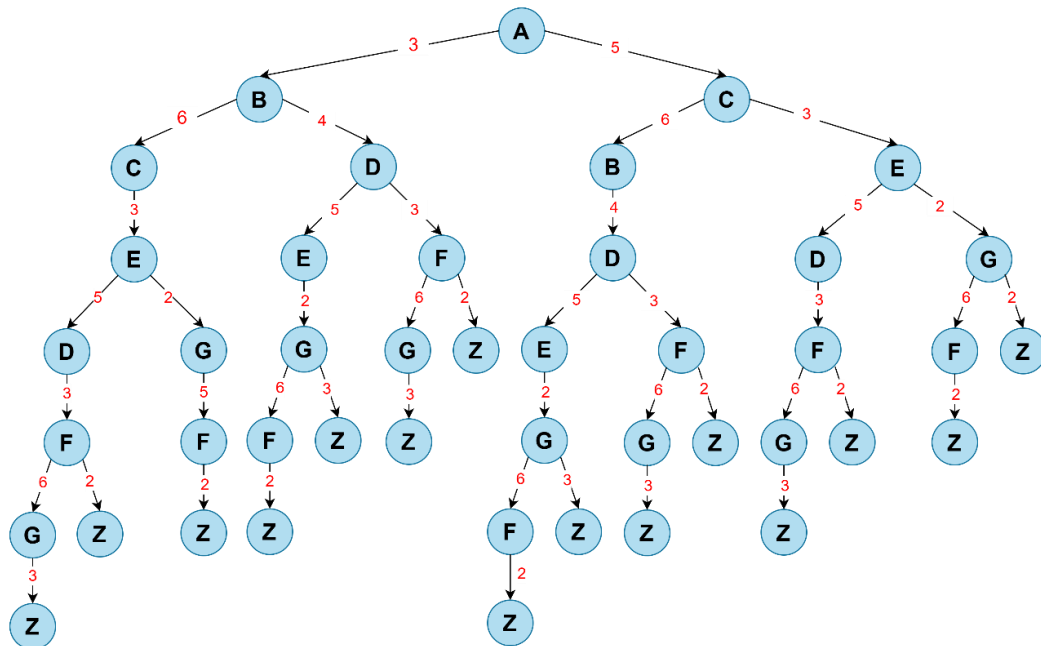
- ``def dijkstra()`` menggunakan algoritma dijkstra untuk menemukan jalur terpendek berdasarkan bobot minimum kumulatif dari start ke goal.
- **Unvisited** (node belum dilalui) adalah direktori yang menyimpan jarak terpendek sementara ke setiap node. Semua node diinisialisasikan dengan `float("inf")`, kecuali start yang diset ke 0.
- **Visited** (node yang suda dikunjungi) direktori yang menyimpan jarak terpendek final ke setiap node setelah dikunjungi.
- **Current Node** adalah node dengan biaya terendah diambil dari unvisited menggunakan ``min`unvisited, key=unvisited.get)`.
- **Cost** adalah bobot dari `current_node` ke `neighbor` digunakan untuk menghitung jarak komulatif ke node lain.
- **Neighbor** adalah node yang terhubung langsung dengan `current_node`. Jika `neighbor` belum dilalui, jaraknya diperbarui saat melalui jalur lebih pendek.
- **Path** adalah direktori yang menyimpan jalur node sebelumnya untuk merekonstruksi jalur terpendek ke goal (Z).
- **Node** mulai dari goal, mundur ke start menggunakan path hingga membentuk jalur terpendek.
- **Final Path** adalah list yang menyimpan urutan jalur terpendek dari start ke goal. Jika jalur tidak ada akan mengembalikan "jalur tidak ditemukan".
- **Dijkstra** adalah algoritma pencarian jalur terpendek berdasarkan cost kumulatif minimum. Node dipilih berdasarkan jarak minimum, dan hasil akhirnya adalah jalur terpendek beserta biayanya.

Hasil Running

```
PS D:\Python\Tugas_KecerdasanBuatan> python tugas1_ppt.py
BFS Path: ['S', 'A', 'D', 'Z']
DFS Path: ['S', 'A', 'B', 'C', 'D', 'Z']
Best First Search Path: ['S', 'B', 'C', 'Z']
Hill Climbing Path: ['S', 'B', 'C', 'Z']
Branch and Bound Path: (['S', 'B', 'C', 'Z'], 11)
Dijkstra Path: (['S', 'B', 'C', 'Z'], 11)
```

b. Tugas di Lampiran

Gambar Diagram Tree



- Breadth First Search

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def bfs(self, start, goal):
        queue = [(start, [start])]
        visited = set()

        while queue:
            node, path = queue.pop(0)
            if node == goal:
                return path

            if node not in visited:
                visited.add(node)
                for neighbor, _ in self.graph[node]:
                    if neighbor not in visited:
                        queue.append((neighbor, path + [neighbor]))

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("BFS:", g.bfs("A", "Z"))
```

Analisa:

- **Class graph** untuk membuat struktu grap tidak berarah menggunakan direktori.
- ``add_edge`` digunakan untuk menambahkan sisi (edge) antara node (u dan v) dengan cost. Karena graf tidak berarah, kedua node saling terhubung.
- ``bfs (start, goal)`` mencari jalur start (A) ke goal (Z) menggunakan breadth first search.
- ``queue = [(start, [start])]` untuk menyimpan node terbaru dan jalur yang ditempuh.
- ``Visited = set()`` untuk menyimpan node yang sudah dilalui agar tidak dilalui kembali. `Visited.add(node)` menandai node bahwa sudah dilalui.
- **While queue** melakukan pencarian hingga antrian kosong atau goal ditemukan.
- **Queue.pop(0)** mengambil node pertama dari antrian.
- ``if node == goal`` maka akan mengembalikan jalur yang dilalui.
- ``for neighbor, _in self.graph[node]:`` untuk iterasi neighbor dari node saat ini. if neighbor not in visited, neighbor di tambahkan ke antrian dengan jalur baru.
- **BFS** (Breadth First Search) efektif untuk mencari jalur terpendek dalam graf tanpa bobot menggunakan queue untuk eksplorasi level demi level.

- Depth First Search

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def dfs(self, start, goal, path=None, visited=None):
        if path is None:
            path = [start]
        if visited is None:
            visited = set()

        if start == goal:
            return path

        visited.add(start)

        for neighbor, _ in self.graph[start]:
            if neighbor not in visited:
                new_path = self.dfs(neighbor, goal, path + [neighbor], visited)
                if new_path:
                    return new_path

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("DFS:", g.dfs("A", "Z"))
```

Analisa:

- ``dfs(self, start, goal, path=None, visited=None)`` adalah pencarian menggunakan algoritma Depth First Search untuk mencari jalur dari start ke goal dengan penandaan visited agar tidak terjadi perulangan. Path untuk menyimpan jalur yang sudah ditempuh, visited untuk menyimpan node yang sudah dilalui agar tidak dilalui dua kali.
- ``path is None : => path = [start]`` untuk menandai jalur awal jika belum dilalui.
- ``visited is None: => visited = set()`` menandai set untuk menyimpan node yang telah dilalui.
- Jika ``start == goal`` maka result akan mengembalikan jalur yang sudah dilalui.
- ``Visited.add(start)`` untuk menandai node terbaru telah dilalui.
- ``if neighbor not in visited``, jika neighbor belum dikunjungi, akan melanjutkan rekursi DFS dengan jalur baru

- ``New_path => return new_path``. Jika jalur ditemukan, akan mengembalikan jalur.
- Best First Search

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def best_first_search(self, start, goal):
        queue = [(0, start, [start])]
        visited = set()

        while queue:
            queue.sort() # Urutkan berdasarkan biaya (heuristik)
            _, node, path = queue.pop(0)

            if node == goal:
                return path

            if node not in visited:
                visited.add(node)
                for neighbor, cost in self.graph[node]:
                    if neighbor not in visited:
                        queue.append((cost, neighbor, path + [neighbor]))

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("Best First Search:", g.best_first_search("A", "Z"))
```

Analisa:

- ``Def best_first_search(self, start, goal)`` adalah algoritma best first search berdasarkan heuristic (biaya).
- ``Queue.sort()`` untuk mengurutkan antrian berdasarkan biaya terendah.
- ``_, node, path = queue.pop(0)`` untuk mengambil node dengan cost terendah dari antrian.
- ``Neighbor, cost in self.graph[node]:`` untuk iterasi neighbor node dan biaya ke neighbor tersebut.
- ``Neighbor not in visited`` akan ditambahkan ke antrian dengan biaya.
- **Best First Search** adalah algoritma pencarian dengan heuristic biaya terendah untuk mencari jalur dari start (A) ke goal (Z).

- Hill Climbing

```

class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def hill_climbing(self, start, goal):
        current = start
        path = [start]
        visited = set([start])

        while current != goal:
            neighbors = sorted(self.graph[current], key=lambda x: x[1])
            found = False
            for neighbor, _ in neighbors:
                if neighbor not in visited:
                    path.append(neighbor)
                    visited.add(neighbor)
                    current = neighbor
                    found = True
                    break

            if not found: # Tidak ada tetangga yang lebih baik, berhenti
                return None

        return path

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("Hill Climbing:", g.hill_climbing("A", "Z"))

```

Analisa:

- **`Def hill\_climbing(self, start, goal)`** untuk implementasi algoritma hill_climbing dengan algoritma pencarian berdasarkan nilai heuristic.
- **`While current != goal:`** adalah loop utama yang berjalan hingga mencapai goal.
- **`neighbors = sorted(self.graph[current], key=lambda x: x[1])`** untuk mengurutkan neighbor berdasarkan bobot terkecil.
- **`Found = False`** adalah variabel untuk menandai apakah ada langkah yang lebih baik.
- **`path.append(neighbor) → visited.add(neighbor) → current = neighbor`** digunakan untuk update jalur, dengan menandai jalur yang sudah dikunjungi, dan pindah ke node berikutnya.
- **`if not found: → return None`** untuk menghentikan algoritma, jika tidak

ada neighbor yang lebih baik.

- **`Return path`** untuk mengembalikan jalur dari start ke goal.
- Branch and Bound

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def branch_and_bound(self, start, goal):
        queue = [(0, start, [start])]
        best_path = None
        best_cost = float("inf")

        while queue:
            cost, node, path = queue.pop(0)

            if node == goal:
                if cost < best_cost:
                    best_cost = cost
                    best_path = path
                continue

            for neighbor, edge_cost in self.graph[node]:
                if neighbor not in path:
                    queue.append((cost + edge_cost, neighbor, path + [neighbor]))
                    queue.sort() # Prioritaskan biaya terendah

        return best_path

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("Branch and Bound:", g.branch_and_bound("A", "Z"))
```

Analisa:

- **`branch\_and\_bound(self, start, goal)`** untuk Implementasi Branch and Bound, algoritma pencarian jalur dengan cost minimum dan mengabaikan jalur yang lebih banyak cost.
- **`best\_path = None, best\_cost = float("inf")`** untuk menyimpan jalur terbaik dengan cost minimum.
- **`cost, node, path = queue.pop(0)`** untuk mengambil jalur dengan cost terendah dari antrian.
- **`while queue:`** merupakan loop utama yang berjalan hingga semua kemungkinan jalur diuji.

- ``queue.append((cost + edge_cost, neighbor, path + [neighbor]))`` untuk menambahkan jalur baru dengan cost yang diperbarui ke antrian.
- ``Return best_path`` untuk mengembalikan jalur terbaik dengan cost terendah.
- Dynamic Programming

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, cost))
        self.graph[v].append((u, cost))

    def dynamic_programming(self, start, goal):
        memo = {}

        def dp(node, path, cost):
            if node == goal:
                return path

            if node in memo:
                return memo[node]

            best_path = None
            for neighbor, edge_cost in self.graph[node]:
                if neighbor not in path:
                    new_path = dp(neighbor, path + [neighbor], cost + edge_cost)
                    if new_path:
                        if best_path is None or len(new_path) < len(best_path):
                            best_path = new_path

            memo[node] = best_path
            return best_path

        return dp(start, [start], 0)

g = Graph()
edges = [
    ("A", "B", 3), ("A", "C", 5), ("B", "D", 4), ("B", "C", 6),
    ("C", "E", 3), ("D", "E", 5), ("D", "F", 3), ("E", "G", 2),
    ("F", "Z", 2), ("F", "G", 6), ("G", "Z", 3)
]
for u, v, cost in edges:
    g.add_edge(u, v, cost)

print("Dynamic Programming:", g.dynamic_programming("A", "Z"))
```

Analisa:

- ``def dynamic_programing(sef, start, goal)`` adalah implementasi Dynamic programming untuk mencaari jalur paling optimal dan menghindari perhitungan berulang sehingga lebih efisien.
- ``memo = {}`` untuk menyimpan hasil perhitungan agar tidak dihitung ulang.
- ``def dp(node, path, cost):`` merupakan fungsi rekursif untuk menjelajahi graf dan menyimpan jalur terbaik,

- ``Node == goal`` akan mengembalikan jalur yang dilalui, jika sampai tempat tujuan.
- ``if node in memo`` untuk memastikan node sudah dihitung atau belum.
- ``best_path = None`` untuk menyimpan jalur terbaik yang ditemukan.
- ``if neighbor not in path`` untuk menghindari looping dengan tidak melalui node yang sudah dilalui.
- ``if best_path is None or len(new_path) < len(best_path):`` untuk menyimpan jalur terpendek yang ditemukan.
- ``memo[node] = best_path`` untuk menyimpan hasil perhitungan agar tidak dihitung ulang.

Hasil Running

```
PS D:\Python\Tugas_KecerdasanBuatan> python tugas2_ethol.py
BFS: ['A', 'B', 'D', 'F', 'Z']
DFS: ['A', 'B', 'D', 'E', 'G', 'F', 'Z']
Best First Search: ['A', 'B', 'D', 'F', 'Z']
Hill Climbing: ['A', 'B', 'D', 'F', 'Z']
Branch and Bound: ['A', 'B', 'D', 'F', 'Z']
Dynamic Programming: ['A', 'B', 'D', 'E', 'G', 'Z']
```

C. Kesimpulan

Kesimpulan perbandingan enam algoritma pencarian:

- **BFS (Breadth-First Search)** efektif untuk mencari jalur terpendek dalam graf tanpa bobot, menggunakan queue untuk eksplorasi level demi level.
- **DFS (Depth-First Search)** lebih cocok untuk eksplorasi mendalam terlebih dahulu, namun tidak selalu menemukan jalur terpendek dan bisa mengalami looping tanpa penandaan visited.
- **Best First Search** menggunakan heuristic untuk memilih jalur yang paling menjanjikan, tetapi tidak selalu menemukan jalur terpendek karena tidak mempertimbangkan total biaya perjalanan.
- **Hill Climbing** mirip dengan Best First Search tetapi selalu memilih neighbor terbaik di setiap langkah, berisiko terjebak di lokal optimum jika tidak ada jalur lebih baik.
- **Branch and Bound** mencari jalur dengan biaya total minimum menggunakan antrian prioritas, tetapi bisa menjadi lambat jika ada banyak kemungkinan jalur.
- **Dynamic Programming** menyimpan hasil perhitungan sebelumnya (memoization) untuk menghindari perhitungan ulang, cocok untuk graf kompleks tetapi bisa memerlukan banyak memori.

Pemilihan algoritma tergantung pada struktur graf, adanya bobot, serta prioritas antara kecepatan, jalur terpendek, atau efisiensi memori.